Unique Paths (medium)

Problem Statement

Given a 2-dimensional grid of size `m x n` (where `m` is the number of rows and `n` is the number of columns), you need to find out the number of unique paths from the top-left corner to the bottom-right corner. The constraints are that you can only move either right or down at any point in time.

Examples

1. **Example 1**:
   - **Input**: 3, 3
   - **Expected Output**: 6
   - **Justification**: The six possible paths are:
     1. Right, Right, Down, Down
     2. Right, Down, Right, Down
     3. Right, Down, Down, Right
     4. Down, Right, Right, Down
     5. Down, Right, Down, Right
     6. Down, Down, Right, Right
2. **Example 2**:
   - Input: 3, 2
   - Expected Output: 3
   - Justification: The three possible paths are:
     - Right, right, down
     - Right, down, right
     - Down, right, right
3. **Example 3**:
   - Input: 2, 3
   - Expected Output: 3
   - Justification: The three possible paths are:
     - Down, right, right
     - Right, down, right
     - Right, right, down

Try it yourself

Try solving this question here:

Java

```java
public class Solution {
    public int uniquePaths(int m, int n) {
        // ToDo: Write Your Code Here.
        return 0;
    }
}

```

Test Case | Run Code Result | Stdout

```
2
2
```

Use Example Testcases | Run | Submit